金融大数据 2026-作业 2

谢铭宸 241275019

2026 年 9 月 6 日

目录

1 简要介绍 SMP、MPP 和 Cluster 各自的特点和区别 **3**

2 在分布存储结构下，如何保证各节点的并行同步计算？ **4**

3 并行系统的可靠性设计与容错技术有哪些具体措施？ **4**

4 MPI 的通信方式有哪些？简要说明同步通信和异步通信的区别。 **5**

5 在单机上安装或基于 Docker 安装并运行 MPICH，尝试改写讲义中的积分示例。 **7**

1 简要介绍 SMP、MPP 和 Cluster 各自的特点和区别

SMP（Symmetric Multi Processing，对称多处理系统）由多个相同类型的处理器组成，这些处理器通常通过总线连接并共享同一个存储器，同时运行一个操作系统。SMP 属于共享内存式并行计算系统，其特点是处理器之间共享数据比较方便，但由于共享总线和共享存储器会逐渐成为性能瓶颈，因此系统扩展能力较弱，规模通常较小。典型规模约为 2–8 个处理器。

MPP（Massively Parallel Processing，大规模并行处理）由多个相互独立的处理节点组成。每个节点拥有独立的处理器、独立的内存和操作系统，各节点之间通常通过专用的高速内部网络进行连接。MPP 的系统规模一般比 SMP 大，适合大规模并行计算，但由于采用专用互联网络和专用体系结构，因此系统升级和扩充相对困难。

Cluster（集群）是由一组商品化计算机或服务器通过网络连接形成的并行计算系统。Cluster 通常使用刀片服务器或机架服务器，并通过网络互联形成一个物理上较紧密的计算系统。与 SMP 和 MPP 相比，Cluster 最大的特点是采用商品化硬件，因此具有较好的可扩展性，系统规模可大可小，也是目前高性能并行计算中非常常用的一种系统形式。

三者的主要区别可以概括为：

1. SMP 采用共享内存和总线互联，耦合程度高，但扩展性较弱，能耗较低；

2. MPP 的节点具有独立的内存和操作系统，并使用专用高速网络互联，其规模和扩展能力强于 SMP；

3.Cluster 则使用商品化服务器和通用或高速网络互联，具有较低的成本和较强的可扩展性，耦合程度较低，能耗较高

补充而言，从并行编程角度看，SMP 通常适合采用 OpenMP、Pthread 等共享内存式并行编程方式；而 MPP 和 Cluster 中的节点通常具有独立内存，因此更适合采用 MPI 等消息传递式并行编程方法。

2 在分布存储结构下，如何保证各节点的并行同步计算？

在分布存储结构中，各个处理器或计算节点拥有自己独立的本地存储器，因此节点之间不能像共享内存系统一样直接访问彼此的数据。为了完成并行计算，**需要首先对计算数据进行划分，然后通过节点之间的数据通信，将相应的数据传送给各个计算节点**。

通常可以由主节点负责数据的划分和分发，各个从节点获得自己的计算数据后分别执行并行计算。由于不同节点的计算速度可能不同，某些节点可能较早完成计算，而另外一些节点仍然处于计算过程中，因此还需要使用同步控制机制保证各节点的执行进度一致。

一种典型的同步机制是 Barrier（同步障）。当一个节点执行到 Barrier 时，如果其他参与计算的节点尚未到达该位置，该节点必须等待；只有当所有相关节点都到达 Barrier 以后，所有节点才能继续执行下一阶段的计算。

在 MPI 程序中，可以使用如下函数实现这种同步：

```
MPI_Barrier(MPI_COMM_WORLD);
```

因此，在分布存储结构下，保证各节点同步计算的主要方法可以概括为：先通过消息传递完成数据的划分和通信，再利用 Barrier 等同步机制协调不同节点的计算进度。

同步操作会使较快的节点等待较慢的节点，因此 Barrier 并不是使用得越多越好。实际程序中通常只在下一阶段必须等待所有节点完成当前阶段时才进行同步，以减少不必要的等待时间并提高并行效率。

3 并行系统的可靠性设计与容错技术有哪些具体措施？

大型并行计算系统中通常包含大量计算机和计算节点，因此节点出错、磁盘故障或任务执行失败是实际运行过程中可能经常遇到的问题。并行系统不能因为一个节点失效就导致数据丢失、整个程序终止或整个系统崩溃，因此必须进行可靠性设计并提供相应的容错和恢复机制。

具体措施主要包括以下两个方面：

1. **数据失效恢复**

   大量计算数据通常存储在多个磁盘或多个节点上。当出现磁盘损坏、节点故障或者数据损坏时，系统需要具有良好的数据备份和数据失效恢复机制，从而保证数据不会因为单个设备失效而丢失，同时保证恢复后数据的完整性和正确性。

2. **系统和任务失效恢复**

   一个计算节点失效时，不能导致整个并行系统崩溃，也不能使整个计算任务永久终止。因此系统需要提供失效检测和失效节点隔离机制，及时发现发生故障的节点并将其从正常计算过程中隔离。对于原来由失效节点承担的计算任务，系统还需要重新进行任务调度，将这些任务分配给其他正常节点继续执行，从而保证整个计算任务能够正常完成。

因此，并行系统的可靠性设计可以概括为：通过数据备份与数据恢复保证数据可靠性，通过节点失效检测、故障隔离和任务重新调度保证系统和计算任务的可靠运行。

在实际的大型并行计算系统中，还常采用 Checkpoint/Restart（检查点与重启）机制。程序运行过程中定期保存计算状态，当某个节点或任务发生故障时，可以从最近一次保存的状态继续计算，从而避免全部任务从头重新执行。

4 MPI 的通信方式有哪些？简要说明同步通信和异步通信的区别。

MPI 主要提供两类通信方式：点对点通信（Point-to-Point Communication）和节点集合通信（Collective Communication）。

1. **点对点通信**

   点对点通信是指两个 MPI 进程之间直接进行消息发送和接收。MPI 的点对点通信又可以分为同步通信（阻塞通信）和异步通信（非阻塞通信）。

   同步通信的典型接口包括：

   ```
   MPI_Send(...)
   ```

```
MPI_Recv(...)
```

同步通信也称为阻塞式通信。当进程调用通信函数以后，必须等待该通信操作完成之后函数才返回，然后程序才能继续执行后面的操作。因此，当通信时间较长时，处理器可能处于等待状态，从而影响系统并行效率。

异步通信的典型接口包括：

```
MPI_Isend(...)
MPI_Irecv(...)
```

异步通信也称为非阻塞式通信。调用通信函数以后，不需要等待数据传输完全结束就可以立即返回，进程可以继续执行其他计算。之后可以使用：

```
MPI_Wait(...)
MPI_Test(...)
```

等函数等待或者检查非阻塞通信是否已经完成。

因此，同步通信和异步通信的主要区别是：同步通信必须等待通信操作完成以后才能继续执行，而异步通信可以在通信尚未结束时继续进行其他计算，从而有机会实现通信过程和计算过程的重叠，减少处理器的空闲等待时间。

2. **节点集合通信**

集合通信是指一个通信组中的多个进程共同参与的数据通信或同步操作。MPI 支持一对多广播、多节点计算同步以及结果规约等功能。常见的集合通信接口包括：

```
MPI_Bcast(...)
MPI_Barrier(...)
MPI_Scatter(...)
MPI_Gather(...)
MPI_Reduce(...)
```

其中,MPI_Bcast 用于广播数据,MPI_Barrier 用于进程同步,MPI_Scatter 用于数据分发，MPI_Gather 用于数据收集，MPI_Reduce 用于对多个进程的计算结果进行求和、求最大值、求最小值等规约操作。

非阻塞通信虽然可以提高并行效率，但在 MPI_Isend 或 MPI_Irecv 真正完成之前，程序需要注意相关通信缓冲区的使用，通常应通过 MPI_Wait 或 MPI_Test 确认通信结束后再安全地使用或修改相关数据。

5 在单机上安装或基于 Docker 安装并运行 MPICH，尝试改写讲义中的积分示例。

本实验在讲义给出的 MPI 并行积分程序基础上进行了适当改写，但保留了原程序的核心并行计算方法，**即利用进程的 rank 对积分区间中的小矩形计算任务进行循环划分，各进程独立计算自己的局部积分结果 `local`，最后使用 `MPI_Reduce` 和 `MPI_SUM` 将所有进程的局部结果求和并发送到 rank 为 0 的进程中**。

讲义中的积分示例计算：

$$\int_0^{10} x^2\,dx$$

其基本思想是将区间 $[0,10]$ 划分成 N 个很小的区间，利用矩形面积之和近似计算积分。设每个小矩形宽度为

$$dx = \frac{b-a}{N},$$

对于第 i 个小区间，取其中点

$$x = a + i \cdot dx + \frac{dx}{2},$$

则该小矩形的面积近似为

$$x^2 \cdot dx.$$

在 MPI 并行程序中，首先使用 MPI_Comm_rank 获得当前进程编号，使用 MPI_Comm_size 获得参与计算的进程总数。不同进程根据自己的 rank 分别计算不同的小矩形。核心任务划分代码为：

```
for (i = myid; i < N; i += numprocs)
{
    x = a + i * dx + dx / 2.0;
    local += x * x * dx;
}
```

其中，local 表示每个进程独立计算得到的局部积分结果。例如，当有 4 个进程时，rank 为 0 的进程负责编号为 $0, 4, 8, \ldots$ 的小矩形，rank 为 1 的进程负责编号为 $1, 5, 9, \ldots$ 的小矩形，其他进程以此类推。

所有进程完成局部计算后，使用 MPI_Reduce 将各进程的 local 值进行求和：

```
MPI_Reduce(&local, &inte, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
```

MPI_SUM 表示对所有进程的 local 进行求和，参数 0 表示最终的规约结果 inte 存放在 rank 为 0 的进程中。最后由 rank 为 0 的进程输出最终积分结果。

在 Docker 环境中可以采用如下方式安装 MPICH：

```
docker run -it --rm ubuntu:24.04 bash
apt update
apt install -y build-essential mpich libmpich-dev
```

安装完成以后可以检查 MPI 环境：

```
mpicc --version
mpirun --version
```

改写后的程序如下：

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
```

```
int myid, numprocs;
long long N = 10000000;

const double a = 0.0;
const double b = 10.0;

MPI_Init(&argc, &argv);

MPI_Comm_rank(MPI_COMM_WORLD, &myid);
MPI_Comm_size(MPI_COMM_WORLD, &numprocs);

if (argc > 1)
    N = atoll(argv[1]);

double dx = (b - a) / N;
double local = 0.0;
double inte = 0.0;

for (long long i = myid; i < N; i += numprocs)
{
    double x = a + i * dx + dx / 2.0;
    local += x * x * dx;
}

MPI_Reduce(&local, &inte, 1,
           MPI_DOUBLE, MPI_SUM,
           0, MPI_COMM_WORLD);

if (myid == 0)
{
    double exact = 1000.0 / 3.0;

    printf("Number of processes: %d\n", numprocs);
```

```
        printf("Numerical integral : %.15f\n", inte);
        printf("Exact value        : %.15f\n", exact);
        printf("Error              : %.15e\n",
               inte > exact ? inte - exact : exact - inte);
    }

    MPI_Finalize();
    return 0;
}
```

程序可以使用 mpicc 进行编译：

```
mpicc -O2 integral.c -o integral
```

例如使用 4 个 MPI 进程运行：

```
mpirun -np 4 ./integral
```

也可以通过命令行指定矩形数量，例如：

```
mpirun -np 4 ./integral 100000000
```

理论上，

$$\int_0^{10} x^2\,dx = \left.\frac{x^3}{3}\right|_0^{10} = \frac{1000}{3} \approx 333.3333333333.$$

因此程序计算出的结果应当接近 333.3333333333。

本实验体现了 MPI 并行计算中典型的 “任务划分–局部计算–结果规约” 过程。各进程首先根据自己的 rank 分别计算一部分小矩形的面积，得到局部结果 local；随后通过 MPI_Reduce 和 MPI_SUM 将所有进程的局部计算结果求和，并将最终结果保存到 rank 为 0 的进程中。与串行计算相比，该方法能够将大量独立的小矩形计算任务分配给多个 MPI 进程并行执行。